

Содержание

1. Решение задачи управления манипулятором в пространстве с препятствиями	2
1.0.1. 2D-случай	2
1.1. Постановка задачи	2
1.2. Движение к заданным точкам, градиентный спуск	2
1.3. Решение на Python	4
1.4. Итеративное приближение к заданным точкам с обходом препятствий, обучение MLP	5

1. Решение задачи управления манипулятором в пространстве с препятствиями

1.0.1. 2D-случай

1.1. Постановка задачи

Пусть манипулятор имеет n сочленений. Обозначим вектор углов поворота сочленений манипулятора: $\theta = (\theta_1, \theta_2, \dots, \theta_n) \in \mathbb{R}^n$ и вектор пар координат сочленений: $J = (J_1, J_2, \dots, J_n) \in (\mathbb{R} \times \mathbb{R})^n$, $J_m = (x_m, y_m)$. Функция решения прямой кинематики для робота с заранее заданными длинами звеньев: $f_{fk} : \mathbb{R}^n \rightarrow (\mathbb{R}^2)^n$, $J = f_{fk}(\theta)$.

Необходимо перевести робота в заданное положение, выполнив обход расположенных в пространстве оперирования робота препятствий.

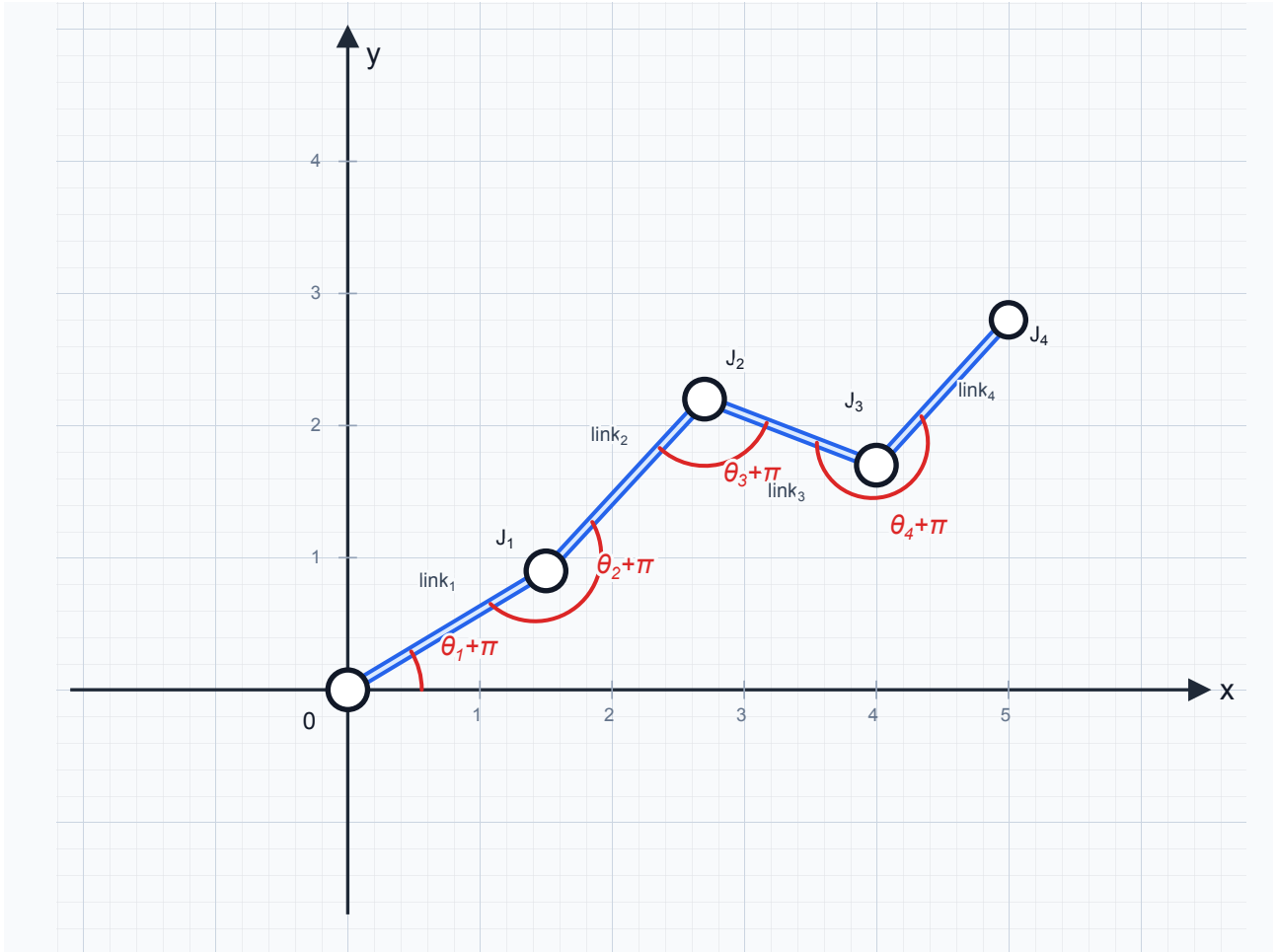


Рисунок 1.1. Схема четырехзвенного робота-манипулятора

1.2. Движение к заданным точкам, градиентный спуск

Обозначим также целевые точки G_m , в которые должны прийти определенные сочленения робота (не все, а, например, только конечные). Индексы $I \subseteq \{1, \dots, n\}$ соответствуют номерам сочленений, к которым относятся целевые точки: $G_m = (x_m^*, y_m^*) \in \mathbb{R}^2$, $m \in I$. И расстояния между целевыми точками и сочленениями: $l_m(\theta) = \|J_m(\theta) - G_m\|_2$, $m \in I$, $L = \{l_m \mid m \in I\}$. Если $m, m+1 \in I$, то $\|G_{m+1} - G_m\|_2 = d_m$, где d_m — длина звена между J_m и J_{m+1} .

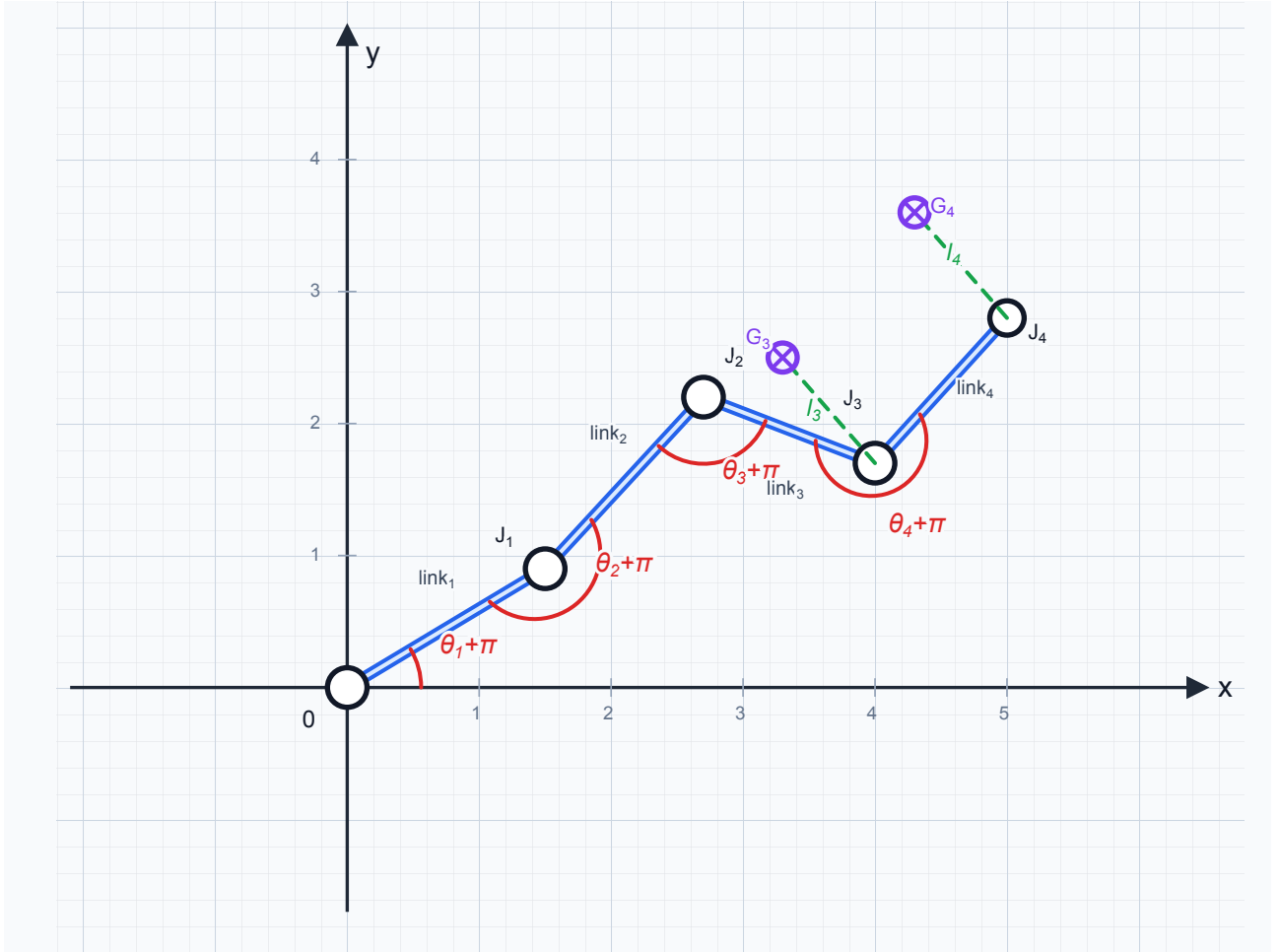


Рисунок 1.2. Схема целевых точек и расстояний для сочленений

Решение задачи регрессии будет заключаться в минимизации функции суммы расстояний:

$$f_{dist} : \mathbb{R}^n \rightarrow \mathbb{R},$$

$$f_{dist}(\theta) = \sum_{i \in I} \|(f_{fk}(\theta))_i - G_i\|_2^2 = \sum_{i \in I} ((f_{fk}(\theta))_i - G_i)^T ((f_{fk}(\theta))_i - G_i)$$

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^n} f_{dist}(\theta)$$

Тогда:

$$\nabla_{\theta} f_{dist}(\theta) = 2 \sum_{i \in I} \left(\frac{\partial (f_{fk}(\theta))_i}{\partial \theta} \right)^T ((f_{fk}(\theta))_i - G_i) = 0 \Rightarrow$$

$$\Rightarrow \sum_{i \in I} \left(\frac{\partial (f_{fk}(\theta))_i}{\partial \theta} \right)^T ((f_{fk}(\theta))_i - G_i) = 0$$

Запишем формулу для прямой кинематики:

$$J_m(\theta) = \sum_{k=1}^m d_k \begin{pmatrix} \cos(\theta_1 + \theta_2 + \dots + \theta_k) \\ \sin(\theta_1 + \theta_2 + \dots + \theta_k) \end{pmatrix}$$

$$f_{fk}(\theta) = (J_1(\theta), J_2(\theta), \dots, J_n(\theta))$$

Отсюда аналитически выразим Якобиан по столбцам:

$$\frac{\partial J_m(\theta)}{\partial \theta_j} = \begin{cases} \sum_{k=j}^m d_k \begin{pmatrix} -\sin \varphi_k \\ \cos \varphi_k \end{pmatrix}, & j \leq m \\ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, & j > m \end{cases}$$

где $\varphi_k = \sum_{r=1}^k \theta_r = \theta_1 + \theta_2 + \dots + \theta_k$

Для решения задачи будем использовать градиентный спуск.

1.3. Решение на Python

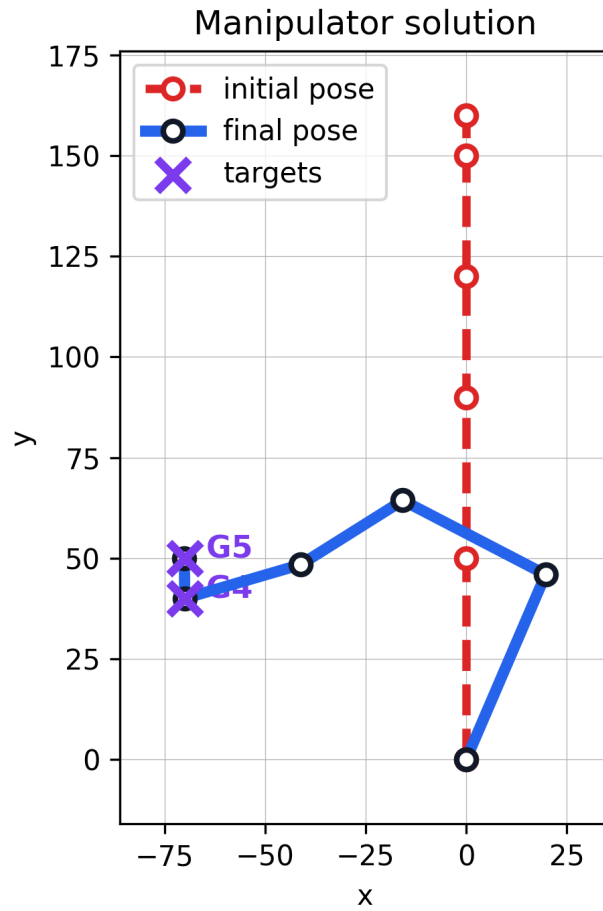


Рисунок 1.3. robot solution

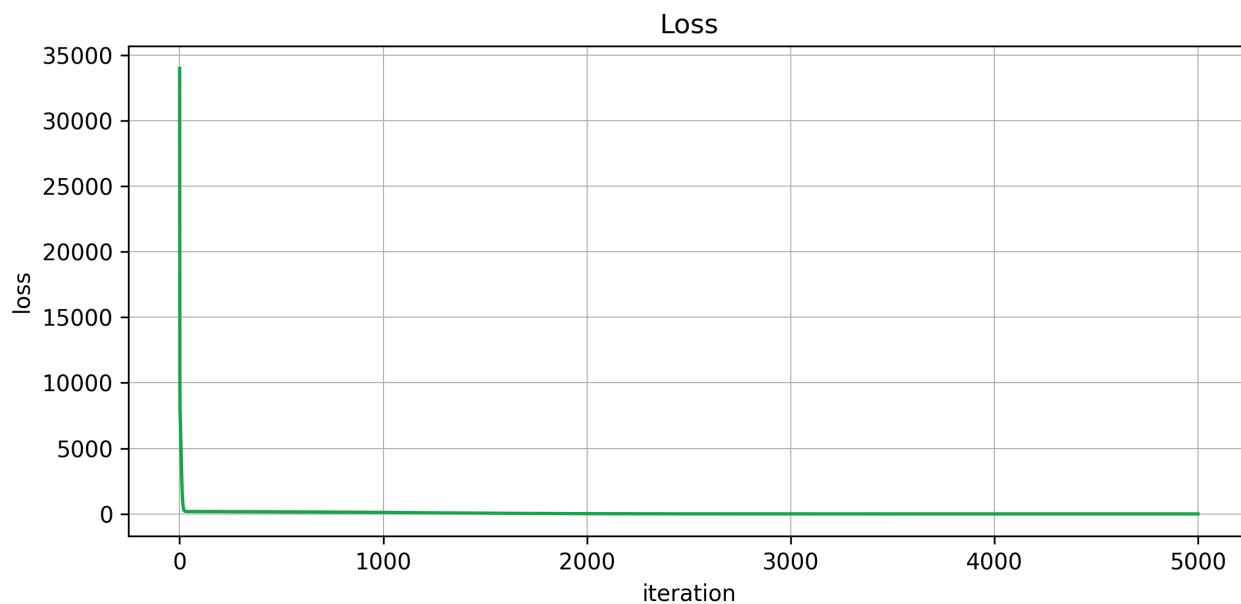


Рисунок 1.4. loss

1.4. Итеративное приближение к заданным точкам с обходом препятствий, обучение MLP

Для обхода препятствий во время движения к целевым точкам разделим перемещение на итерации. На каждой итерации генерируется облако точек для каждого сочленения J_i : $C^i = \{c_1^i, c_2^i, \dots, c_n^i\}$, $c_k^i \in \mathbb{R}^2$, область для генерации задаётся кольцом с внутренним радиусом r_{in} и внешним r_{out} .

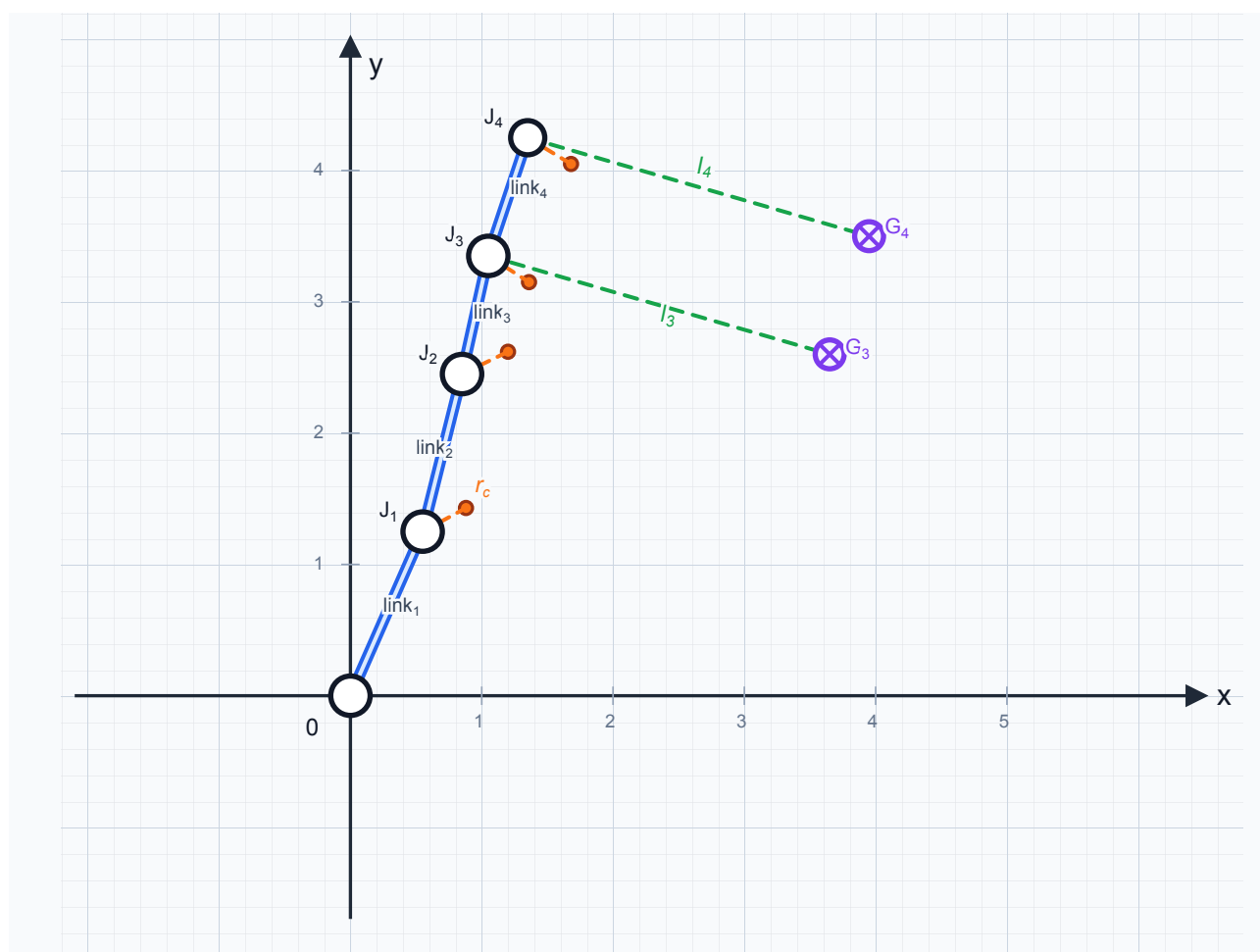


Рисунок 1.5. Итерация